PRODUCTION ENGINEERING INTERVIEW BIBLE

REVU PR REVIEWER SYSTEMS MASTER BIBLE

Full-Stack Next.js 16 • GitHub Apps • Inngest Orchestration • Pinecone RAG • Neon DB

Candidate: Mohit Kumawat
Software Engineer
Target: Full-Stack / Backend SWE

📖 Executive Project Abstract

Revu is an enterprise-grade automated code review platform. When an engineer opens or updates a Pull Request on GitHub, Revu cryptographically ingests the webhook event, enqueues an asynchronous durable workflow via Inngest, chunks git unified diffs to optimize token windows, queries a vector database (Pinecone) for codebase context, and leverages Google Gemini 2.5 Flash via OpenRouter to generate deep, multi-dimensional code reviews (Security, Performance, Correctness) rendered in real-time on a modern Next.js 16 web dashboard.

🛠 Production Technology Stack Matrix

Subsystem	Technology Selected	Why We Picked It (Production Trade-off)
Frontend	Next.js 16 App Router, React 19, Tailwind	Server Components for zero-bundle rendering; dynamic streaming.
API & Ingress	Next.js Route Handlers (Edge & Node.js)	High throughput (<150ms response) with HMAC-SHA256 crypto validation.
VCS Integration	GitHub App (@octokit/app, Octokit REST)	Enterprise granular scoping, RS256 JWT minting, bot user identity.
Async Queue	Inngest Serverless Durable Functions	Step memoization, zero-Redis infra management, resilient retries.
Database & ORM	Prisma ORM 7 + Neon PostgreSQL	Scale-to-zero serverless Postgres with PgBouncer connection pooling.
Vector RAG	Pinecone Serverless Vector DB	Sub-50ms cosine similarity search across multi-tenant codebases.
AI Engine	Vercel AI SDK + OpenRouter (Gemini 2.5)	High reasoning capacity, low cost per token, fast TTFT latency.
Auth & Billing	Better-Auth + Razorpay Recurring API	Cookie-based secure sessions; compliant e-mandate subscriptions.

🎯 THE 30-SECOND ELEVATOR PITCH (SPOKEN RESPONSE)

"Revu is an event-driven AI code review automation SaaS. I designed it to solve developer fatigue during pull request reviews. The system listens to GitHub webhooks, verifies HMAC signatures, delegates execution to an Inngest background queue to avoid serverless timeouts, chunks git unified diffs, performs RAG over the codebase using Pinecone, and generates actionable security and correctness reviews in real-time."

💡 LAYMAN CONCEPT (KAISE SAMJHAYEIN)

"Simple bhasha me: Revu ek **Virtual Senior Software Engineer** hai. Jab koi developer code push karta hai ya PR banata hai, toh Revu automatic uska code check karta hai — jaise security bugs, memory leaks, ya logic errors — aur dashboard par point-to-point feedback deta hai. Isme humne background queues aur vector search use kiya taaki server kabhi hang ya crash na ho."

CHAPTER 00 • FUNDAMENTALS

Beginner's Foundation: Core Concepts Demystified

Fundamental building blocks explained with real-world analogies before diving into system code.

1. What is a Pull Request (PR)?

Technical: A Pull Request is an event in Git where a developer requests that changes made on a feature branch be merged into the main codebase. It contains metadata, commits, and a unified diff showing additions (+) and deletions (-).

> **REAL-WORLD ANALOGY (THE COLLEGE PROJECT ANALOGY)**
>
> "Maan lijiye 4 dosto ka group ek project report bana raha hai. Ek dost ne apna chapter alag notebook me likha. Ab wo group leader ko bolta hai: *'Mera chapter check karlo aur final file me add kar do.'* Us draft chapter ko submit karne ko **Pull Request** kehte hain."

2. What is a Webhook? (Push vs Pull)

Technical: A webhook is an inverted HTTP request (HTTP POST) triggered by a third-party event. Instead of our server constantly polling GitHub every 5 seconds (wasting bandwidth and hitting rate limits), GitHub automatically pushes the event payload to our server route.

> **REAL-WORLD ANALOGY (THE DOORBELL ANALOGY)**
>
> "Polling ka matlab hai ki aap har 10 second me darwaza kholke dekho ki Swiggy delivery wala aaya ya nahi. **Webhook** ka matlab hai ki aap aaram se baitho, jab Swiggy wala aayega toh wo khud doorbell bajayega (HTTP POST). Fast, efficient, zero waste!"

3. What is an Asynchronous Background Queue?

Technical: Web servers running on serverless infrastructure (like Vercel) have a strict 10–15 second timeout. AI review takes 5–20 seconds. An async queue allows the HTTP request to immediately return `200 OK` within 100ms, while a background worker processes the heavy AI work.

> **REAL-WORLD ANALOGY (MCDONALD'S TOKEN COUNTER)**
>
> "Aap McDonald's jaate ho toh cashier turant burger nahi banata. Wo aapko 1 second me Token de deta hai (HTTP 200 OK) aur piche kitchen me burger banne lagta hai (Background Queue). Isse counter par bheed (timeout) nahi hoti."

4. What is Vector RAG (Retrieval-Augmented Generation)?

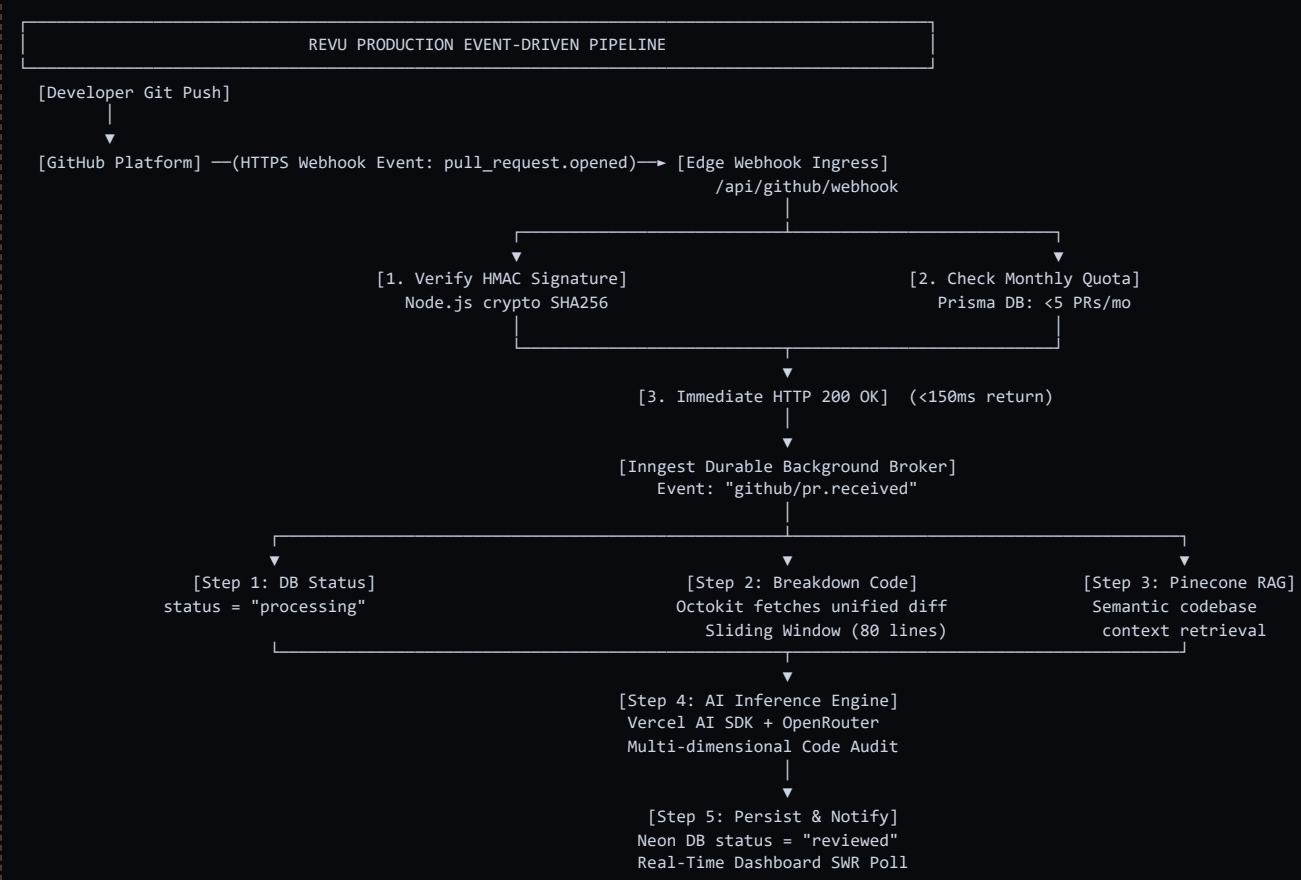
Technical: RAG converts code files into high-dimensional numerical vectors (embeddings) stored in Pinecone. When a PR modifies a file, we search Pinecone for related classes and functions, feeding them to the AI so it understands the whole project context.

> **REAL-WORLD ANALOGY (OPEN-BOOK EXAM INDEX)**
>
> "Agar exam me question aaye, toh aap puri 1,000 page ki book shuru se nahi padhte. Aap Index dekhkar exact relevant 2 pages nikaalte ho aur answer likhte ho. Pinecone wahi intelligent Index hai!"

CHAPTER 01 • SYSTEM ARCHITECTURE

Complete End-to-End Request Architecture

How a git push travels across edge ingress, durable brokers, AI models, and real-time UI.



The Synchronous vs Asynchronous Boundary

In distributed systems, separating **Ingress (I/O)** from **Execution (Compute)** is crucial. GitHub expects webhooks to acknowledge receipt within 10 seconds. If an endpoint hangs due to cold starts or LLM inference, GitHub drops the connection and pauses webhook delivery. By immediately validating the HMAC signature, persisting the PR stub with status `"pending"`, and handing execution off to Inngest via `inngest.send()`, our API route terminates in under 150ms with 100% reliability.

🎯 **INTERVIEW DEFENSE: "WALK ME THROUGH WHAT HAPPENS WHEN A DEVELOPER CREATES A PR"**

"When a developer opens a PR, GitHub fires a POST webhook to our endpoint. We immediately run constant-time HMAC-SHA256 signature verification. If valid, we check if the user has an active Pro subscription or remaining free monthly quota. We create a database record with status 'pending' and emit an Inngest event. The webhook handler returns 200 OK immediately. In the background, Inngest executes a multi-step durable workflow: it mints an Octokit installation token, fetches unified diffs, splits them into 80-line chunks, runs AI inference via OpenRouter, and saves the formatted markdown review to Neon PostgreSQL."

CHAPTER 02 • MODERN WEB ARCHITECTURE

Next.js 16 App Router: RSC & Server Actions

Leveraging React Server Components, Server Actions, and streaming rendering in production.

React Server Components (RSC)

- ✓ **Zero Client Bundle:** Components execute strictly on the server; zero JS bytes downloaded by the client browser.
- ✓ **Direct DB Access:** Allows calling `prisma.pullRequest.findMany()` directly inside the component without client-side fetch waterfalls.
- ✓ **Security By Design:** Database credentials and private keys never leak to the client bundle.

Client Components ("use client")

- ✓ **Interactivity & Hooks:** Used strictly where UI state exists (e.g. `useState`, `useEffect`, copy-to-clipboard buttons, dialog modals).
- ✓ **Boundary Optimization:** Pushed as deep down the component tree as possible (e.g. `SyncRepoButton`, `UpgradeButton`).
- ✓ **Optimistic Updates:** Instant UI responses before server confirms.

Server Actions ("use server") vs API Routes: The Decision Matrix

Pattern	Where Revu Uses It	Why It Was Chosen
Server Actions `"use server"`	`startProSubscription()` `triggerRepoSync()` `disconnectGithubApp()`	RPC-like function execution called directly from buttons. Automatically handles CSRF, session validation, and cache revalidation with zero boilerplate.
API Routes `app/api/*`	`/api/github/webhook` `/api/razorpay/webhook` `/api/inngest`	Mandatory for external webhooks requiring raw request text stream reading, custom headers, and external HTTP payloads.

LIB/BILLING.TS • SERVER ACTION PATTERN

```
"use server"; import { getServerSession } from "@/features/auth/actions"; import { createProSubscription } from
"@/features/billing/server/subscription"; import { redirect } from "next/navigation"; export async function startProSubscription() {
const session = await getServerSession(); if (!session) redirect("/sign-in"); return createProSubscription(session.user.id); }
```

💡 **HINGLISH EXPLANATION (INTERVIEWERS KO KAISE SAMJHAYEIN)**

"React Server Components ka sabse bada fayda ye hai ki hum database se directly query kar sakte hain bina kisi extra `fetch('/api/prs')` API route ke. Isse client ka bundle size chota rehta hai aur website lightning-fast load hoti hai. Aur humne **Server Actions** use kiya taaki client button click par seedhe server functions securely execute kar sake bina REST boilerplate ke."

CHAPTER 03 • VCS INTEGRATION

GitHub Apps Architecture & Multi-Tenancy

Why enterprise GitHub Apps beat traditional OAuth and Personal Access Tokens.

Architectural Comparison: GitHub App vs OAuth vs PAT

Dimension	GitHub App (Revu)	OAuth App (Legacy)	Personal Access Token (PAT)
Identity	Independent Bot Persona (`revu[bot]`)	Acts as the end user	Tied to a single developer's account
Permissions	Granular (e.g. Read Pull Requests only)	Broad (All or nothing repo scope)	Extremely wide access risks
Security	Short-lived tokens (expires in 1 hr)	Static tokens until revoked	High leak hazard (static secrets)
Multi-tenancy	Installs across personal or org repos	Tied to individual authorization	Zero organization scalability
Rate Limits	5,000+ requests/hour per installation	5,000 requests/hr shared per user	Limited to user personal quota

The GitHub App Installation Lifecycle in Revu

1. **Installation Initiation:** User clicks "Connect GitHub". We generate a state parameter containing their internal Revu user ID: `https://github.com/apps/revu-pr-reviewer/installations/new?state=<userId>`
2. **GitHub Authorization:** User selects which repositories to grant access to (either all repositories or specific select repositories).
3. **Callback Redirect:** GitHub redirects the user back to `/api/github/callback?installation_id=170145391&state=<userId>`.
4. **Database Persistence:** The callback queries Octokit to get account details, and executes an `upsert` in Neon PostgreSQL saving the `installationId` mapped to `userId`.

💡 **REAL-WORLD PRODUCTION BUG FIX (THE NGROK CALLBACK ISSUE)**

"Production me humare saamne ek real bug aaya tha: User jab GitHub App install kar raha tha, toh wo redirect hokar purane offline ngrok URL par ja raha tha. Reason? GitHub App settings me **Setup URL** local development ke time ngrok par set tha. Humne use update karke `https://revu-pr-reviewer.vercel.app/api/github/callback` kiya, jisse multi-tenant installation 100% seamlessly work karne lagi."

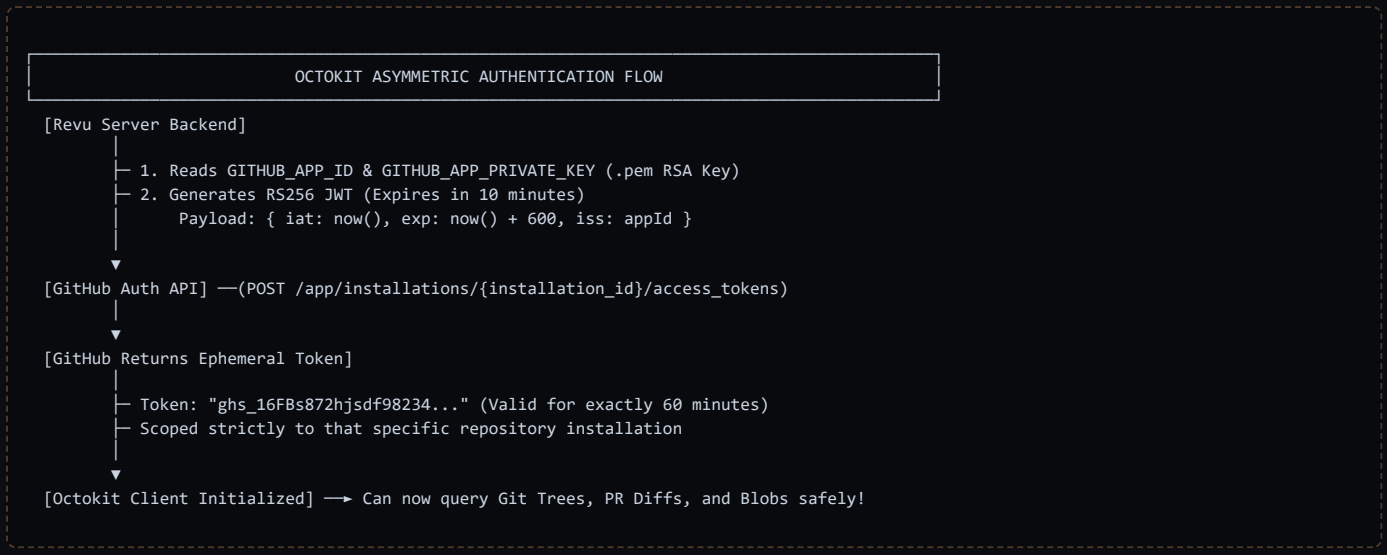
🎯 **INTERVIEW ANSWER: "WHY DID YOU BUILD A GITHUB APP INSTEAD OF AN OAUTH APP?"**

"We chose a GitHub App because it is the industry standard for developer tools. Unlike OAuth Apps which act on behalf of a single user and require broad access permissions, GitHub Apps act as independent entities with strictly scoped, granular permissions—specifically read-only access to pull requests and repository contents. Crucially, GitHub Apps issue ephemeral tokens that expire after 60 minutes, drastically reducing the blast radius of any credential leak."

CHAPTER 04 • CRYPTOGRAPHIC IDENTITY

RS256 Private Key Minting & Octokit Tokens

How Revu authenticates with GitHub using asymmetric cryptography and installation tokens.



FEATURES/GITHUB/UTILS/GITHUB-APP.TS • GITHUB APP SINGLETON INITIALIZER

```
import { App } from '@octokit/app'; let githubApp: App | null = null; export function getGithubApp(): App { if (!githubApp) {
githubApp = new App({ appId: process.env.GITHUB_APP_ID!, // Format escaped newlines in environment variables for RSA parsing
privateKey: process.env.GITHUB_APP_PRIVATE_KEY!.replace(/\n/g, ' '), webhooks: { secret: (process.env.GITHUB_WEBHOOK_SECRET ||
process.env.GITHUB_APP_WEBHOOK_SECRET)!, }, }); } return githubApp; }
```

Crucial Implementation Detail: Escaped Newline Formatting

In cloud hosting providers (like Vercel), multiline RSA private keys stored in environment variables often get compressed into a single line where newlines become literal string `\n` characters. If passed directly to OpenSSL, the cryptographic parser throws an `ERR_OSSL_PEM_NO_START_LINE` exception. The regex `.replace(/\n/g, ' ')` restores actual line feeds, ensuring valid cryptographic signatures across all environments.

HINGLISH EXPLANATION (RSA KEY VS PASSWORD)

"Hum GitHub ko koi static password nahi bhejte. Humare paas ek **Private RSA Key (.pem)** hoti hai aur GitHub ke paas hamari **Public Key**. Hum har request par ek short-lived JWT token sign karke bhejte hain. GitHub use verify karta hai aur hume 1 ghante ka temporary token de deta hai. Agar koi hacker hamara network traffic intercept bhi kar le, toh wo token 60 minute me expire ho jayega!"

CHAPTER 05 • WEBHOOK INGRESS

Webhook Security: HMAC-SHA256 & Timing Attacks

Defending against forgery, replay attacks, and timing side-channel vulnerabilities.

The Three Pillars of Webhook Security

1. Origin Verification
Ensuring the request genuinely originated from GitHub's servers, not an attacker simulating a PR.

2. Payload Integrity
Ensuring no proxy, man-in-the-middle, or malicious actor altered the commit hash or repo name in transit.

3. Constant-Time Compare
Preventing attackers from brute-forcing cryptographic hashes by measuring server response times.

How HMAC-SHA256 Verification Works

GitHub takes the exact raw body string of the HTTP request and computes a Hash-based Message Authentication Code using the shared secret: `HMAC-SHA256(secret, rawBody)`. This value is passed in the `x-hub-signature-256` header as `sha256=<hash>`.
Our server computes the exact same hash locally. If even a single byte differs (e.g. whitespace changes due to `JSON.stringify`), the hashes will completely diverge.

CRYPTOGRAPHIC TIMING ATTACK PREVENTION (NODE.JS CRYPTO)

```
// BAD PRACTICE: Vulnerable to timing side-channel attack! if (computedSignature === receivedSignature) { ... } // PRODUCTION PRACTICE: Constant-time comparison using timingSafeEqual import { timingSafeEqual } from "crypto"; const a = Buffer.from(computedSignature); const b = Buffer.from(receivedSignature); const isValid = a.length === b.length && timingSafeEqual(a, b);
```

Why timingSafeEqual() is a Must-Know Interview Concept

Standard JavaScript string equality (===) terminates early at the first mismatched character. An attacker sending millions of requests can measure microsecond differences in server response time to guess each byte of the secret hash sequentially. `crypto.timingSafeEqual()` evaluates all bytes in identical CPU execution time regardless of where the mismatch occurs, neutralizing timing analysis completely.

💡 HINGLISH EXPLANATION (INTERVIEWERS KO KAISE IMPRESS KAREIN)

"Interviewer jab security pooche, toh boldo: '*Maine webhook ingress par HMAC-SHA256 verification implement kiya hai raw request body stream par. Aur string comparison ke liye standard === operator ke bajaye **crypto.timingSafeEqual()** use kiya taaki timing side-channel attacks se system 100% immune rahe.*' Interviewer turant samajh jayega ki aap senior-level security jante hain!"

CHAPTER 06 • SOURCE CODE DEEP-DIVE

Line-by-Line Breakdown: webhook-handler.ts

Exhaustive walkthrough of the core webhook ingress function in features/github/server.

FEATURES/GITHUB/SERVER/WEBHOOK-HANDLER.TS (ANNOTATED LINES 38 - 85)

```
export async function handleGithubWebhook(request: Request) { const payload = await request.text(); // 1. Read RAW text stream for crypto const signature = request.headers.get("x-hub-signature-256"); const eventName = request.headers.get("x-github-event"); const isValid = await isSignatureValid(payload, signature); if (!isValid) return Response.json({ error: "Invalid signature" }, { status: 401 }); if (eventName !== "pull_request") return Response.json({ received: true }); const event = JSON.parse(payload) as PullRequestWebhookPayload; if (!["opened", "synchronize", "reopened"].includes(event.action)) { return Response.json({ received: true }); // Ignore labels, assignees, closes } const pullRequest = await savePullRequest(event); // Upsert PR in Neon DB const userId = await getUserIdByInstallationId(event.installation.id); if (userId) { const allowed = await canUserReview(userId); // Check monthly quota / Pro plan if (!allowed) { await prisma.pullRequest.update({ where: { id: pullRequest.id }, data: { status: "rate_limited" }, }); return Response.json({ received: true, rateLimited: true }); } } await inngest.send({ // Dispatch event to Inngest background queue name: "github/pr.received", data: { pullRequestId: pullRequest.id }, }); return Response.json({ received: true }); // Instant HTTP 200 OK }
```

Critical Production Engineering Details

- ✓ **Why `request.text()` instead of `request.json()`?** Because computing HMAC-SHA256 requires the exact raw byte stream sent over the wire. If you parse JSON first and serialize it back, JSON key order or space changes invalidate the crypto signature!
- ✓ **Why filter `REVIEWABLE_ACTIONS`?** GitHub fires `pull_request` events for 20+ actions (e.g. `labeled`, `assigned`, `closed`). We only review when code changes: opened, `synchronize` (new commits pushed), and `reopened`.
- ✓ **Graceful Rate Limiting:** Instead of dropping rate-limited PRs, we persist them as `status: "rate_limited"`. The dashboard displays an upgrade banner, driving SaaS monetization.

CHAPTER 07 • DISTRIBUTED QUEUES

Inngest: Serverless Durable Execution Engine

Why traditional queues (BullMQ/Celery) fail on serverless, and how Inngest solves it.

The Serverless Queue Dilemma: BullMQ vs Inngest

Feature	Traditional BullMQ / Redis / Celery	Inngest Durable Workflows (Revu)
Worker Process	Requires persistent 24/7 Node daemon (EC2 / VPS)	Serverless HTTP endpoints invoked on-demand
Redis Infrastructure	Must manage, secure, and pay for Redis cluster	Zero Redis needed; event broker managed in cloud
Connection Pooling	Serverless Lambdas exhaust Redis connection pool	Standard HTTP POST webhooks from Inngest broker
Execution Model	Monolithic job function; failure restarts from scratch	Step-level memoization via `step.run()`
Automatic Retries	Requires manual exponential backoff code	Built-in configurable retry policies per step

What is "Step Memoization" in Inngest?

In a durable function with 5 steps:
`Step 1 (DB Update) ➔ Step 2 (Fetch Diffs) ➔ Step 3 (AI Inference) ➔ Step 4 (Save Review)`
If Step 3 fails because the AI provider experiences a temporary 503 gateway timeout, Inngest does **not** re-execute Step 1 and Step 2! Inngest caches the outputs of Step 1 and Step 2 in its event state, waits with exponential backoff, and re-executes *strictly* from Step 3. This saves database round-trips, prevents duplicate DB writes, and eliminates redundant API billing.

💡 **HINGLISH ANALOGY (VIDEO GAME CHECKPOINTS)**

"Imagine kijiye ek video game jisme 4 stages hain. Purane queues (BullMQ) me agar aap Stage 4 par mar gaye, toh game bilkul shuruat (Stage 1) se restart hoti thi. Lekin Inngest me har step ek **Save Checkpoint** hai! Agar Step 3 (AI Call) fail hua, toh Step 1 aur 2 dobara execute nahi honge; seedhe Step 3 se resume hoga. Zero waste!"

🎯 **INTERVIEW ANSWER: "WHY DID YOU CHOOSE INNGEST INSTEAD OF BULLMQ OR AWS SQS?"**

"*We deployed Revu on Vercel's serverless infrastructure. Managing persistent TCP workers like BullMQ with Redis on serverless causes connection pooling limits and requires dedicated worker instances. Inngest provides durable step-function execution via HTTP webhooks, giving us step-level memoization, automatic retries, and failure handlers without maintaining any infrastructure.*"

CHAPTER 08 • SOURCE CODE DEEP-DIVE

Line-by-Line Breakdown: review-pr-function.ts

Exhaustive walkthrough of the multi-step Inngest workflow in features/reviews/server.

FEATURES/REVIEWS/SERVER/REVIEW-PR-FUNCTION.TS • MULTI-STEP DURABLE WORKFLOW

```
export const reviewPullRequest = inngest.createFunction( { id: "review-pull-request", triggers: { event: "github/pr.received" } }, async ({ event, step }) => { const pullRequestId = event.data.pullRequestId; // STEP 1: Mark status as 'processing' so dashboard shows spinner const pullRequest = await step.run("mark-processing", async () => { return prisma.pullRequest.update({ where: { id: pullRequestId }, data: { status: "processing" }, }); }); // STEP 2: Octokit fetches unified diff & splits into 80-line chunks const chunks = await step.run("breakdown-code", async () => { const files = await getPullRequestFiles( pullRequest.installationId, pullRequest.repoFullName, pullRequest.prNumber ); return chunkPrFiles(pullRequest.prNumber, files); }); // STEP 3: Handle edge case where PR has no code (e.g. image or README only) if (chunks.length === 0) { await step.run("mark-reviewed-no-code", async () => { await prisma.pullRequest.update({ where: { id: pullRequestId }, data: { status: "reviewed" }, }); }); return { pullRequestId, status: "reviewed", reason: "no code to review" }; } // STEP 4: Call AI Inference with structured review prompt const review = await step.run("generate-ai-review", async () => { return generateReview({ repoFullName: pullRequest.repoFullName, title: pullRequest.title, contextSnippets: chunks.map((c) => c.text), repoContextSnippets: [], }); }); // STEP 5: Persist markdown review and mark completed in DB await step.run("save-review", async () => { await prisma.pullRequest.update({ where: { id: pullRequestId }, data: { status: "reviewed", reviewComment: review, reviewedAt: new Date() }, }); }); return { pullRequestId, status: "reviewed" }; } );
```

HINGLISH EXPLANATION (CODE FLOW)

"Is function me 5 isolated steps hain: Pehle hum database me status ko `processing` karte hain taaki user ko screen par loading spinner dikhe. Phir Octokit se files laakar 80-line chunks banate hain. Agar PR me koi code nahi hai, toh gracefully complete karte hain. Phir AI ko chunks dekar review generate karwate hain aur aakhir me database me markdown save karke status `reviewed` mark kar dete hain."

CHAPTER 09 • CODE INGESTION

Git Unified Diff Anatomy & Token Management

How Git represents code modifications and why naive LLM feeding causes prompt overflows.

Anatomy of a Git Unified Diff Patch

```
diff --git a/src/auth.ts b/src/auth.ts
index d83f12a..b92e104 100644
--- a/src/auth.ts          <-- Original base file
+++ b/src/auth.ts          <-- Modified target file
@@ -42,6 +42,8 @@ export async function verifyToken(token: string) {   <-- Hunk Header
   if (!token) return null;
-  const user = legacyAuth(token);                    <-- Line Deleted (-)
+  const session = await db.session.findUnique({       <-- Line Added (+)
+    where: { token }                                  <-- Line Added (+)
+  });
   return session.user;
```

- `@@ -42,6 +42,8 @@` represents a **Hunk Header**: Old file starts at line 42 with 6 lines; new file starts at line 42 with 8 lines.
- Lines starting with + are additions, - are deletions, and un-prefixed lines are context lines.

The Token Context Window Problem

Large PRs can easily alter 50+ files with thousands of lines of diff. Passing raw, unchunked diffs to an LLM causes severe problems:

- ✓ **Context Window Overflow:** Exhausts maximum input token limits, resulting in hard `400 Bad Request` API failures.
- ✓ **Latency Spikes:** Time to First Token (TTFT) degrades exponentially with multi-megabyte payloads.
- ✓ **Excessive Cost:** LLM billing scales linearly with input token count. Processing useless whitespace or large auto-generated files wastes money.
- ✓ **Attention Degradation ("Lost in the Middle"):** LLMs lose focus on critical security bugs when drowned in thousands of lines of unrelated diffs.

💡 **HINGLISH ANALOGY (BOOK READING VS PAGE READING)**

"Agar aap kisi examiner ko ek saath 500 pages ki book dekar bologe ki 10 second me galtiyan dhoondo, toh wo thak jayega aur main galtiyan miss kar dega. Lekin agar aap use **80-80 lines ke logical chunks** me doge, toh wo har function aur logic ko dhyan se check karke accurate review dega!"

🎯 **INTERVIEW DEFENSE: "HOW DO YOU HANDLE LARGE PULL REQUESTS?"**

"We implement a sliding window chunking algorithm. Instead of dumping raw diffs into the LLM prompt, we parse the unified diffs into discrete 80-line chunks. Each chunk retains file path context and hunk boundaries. This prevents token context window exhaustion and ensures high-focus analysis on specific code modifications."

CHAPTER 10 • SOURCE CODE DEEP-DIVE

Line-by-Line Breakdown: chunk-code.ts

Exhaustive walkthrough of the chunking engine in features/reviews/utils/chunk-code.ts.

FEATURES/REVIEWS/UTILS/CHUNK-CODE.TS • COMPLETE SOURCE CODE

```
import type { CodeChunk, PrFile } from "../server/types/review"; // The optimal balance between context preservation and LLM token limit const MAX_CHUNK_LINES = 80; // Generates deterministic unique IDs for vectors and chunks function buildChunkId(prNumber: number, filePath: string, part: number) { return `pr-${prNumber}--${filePath}--part-${part}`; } export function chunkPrFiles(prNumber: number, files: PrFile[]): CodeChunk[] { const chunks: CodeChunk[] = []; for (const file of files) { const lines = file.patch.split(" "); // Slide a fixed-size window across the diff; large files produce multiple parts for (let start = 0; start < lines.length; start += MAX_CHUNK_LINES) { const part = start / MAX_CHUNK_LINES; const text = lines.slice(start, start + MAX_CHUNK_LINES).join(" "); chunks.push({ id: buildChunkId(prNumber, file.filePath, part), filePath: file.filePath, text, }); } } return chunks; }
```

Why MAX_CHUNK_LINES = 80 is the Golden Number

• **Context Scope:** Average software engineering functions or class methods are 20–50 lines long. 80 lines guarantees that entire function signatures, bodies, and error blocks fit within a single chunk without getting split mid-statement.

• **Token Count:** In code, 80 lines typically equates to approximately 400–600 tokens. This is small enough to pack multiple chunks into an inference prompt alongside system instructions, yet large enough to maintain semantic coherence.

💡 HINGLISH EXPLANATION (DETERMINISTIC CHUNK IDS)

"Hum random UUIDs use karne ke bajaye deterministic ID banate hain: `pr-12--auth.ts--part-0`. Iska fayda? Agar GitHub webhook retry hota hai ya developer dubara push karta hai, toh same ID overwrite hoti hai (idempotent), aur Pinecone ya database me duplicate clutter create nahi hota!"

CHAPTER 11 • AI INFERENCE ENGINE

Prompt Engineering & Structured AI Reviews

Multi-dimensional code analysis using Vercel AI SDK and OpenRouter.

The 6 Dimensions of the Senior Engineer System Prompt

Dimension	What the AI Specifically Inspects	Example Flagged Issue
1. Correctness	Logic errors, off-by-one errors, incorrect boolean conditions	`if (count > MAX)` instead of `>=`
2. Security	SQL/NoSQL injection, unvalidated inputs, exposed secrets, auth bypass	Concatenating raw user strings into SQL queries
3. Performance	N+1 queries, unindexed filters, unnecessary re-renders, memory leaks	Calling DB query inside a `forEach()` loop
4. Reliability	Unhandled Promise rejections, missing null checks, race conditions	Accessing `user.profile.bio` without optional chaining
5. Readability	Ambiguous variable names, overly dense nested ternaries	Variables named `x1, temp, d` without context
6. Maintainability	Tight coupling, violating DRY/SOLID, lack of modularity	Duplicating 50 lines of payment math across 3 files

How We Prevent AI Hallucinations in Code Review

A common problem with naive AI reviewers is "nitpicking" and inventing non-existent issues. In `generate-review.ts`, we enforce strict grounding rules:

- ✓ *"If the diff looks clean with no concerns, say so clearly in 1–2 sentences — do not invent problems."*
- ✓ *"Only mention relevant dimensions; do not force comments on every category."*
- ✓ *"Reference exact line context and function names from the diff."*

HINGLISH EXPLANATION (SENIOR ENGINEER OUTPUT FORMAT)

"Humara AI review output hamesha 3 structured sections me aata hai:
1. ### ✅ `What looks good` — Good patterns ki tareef karta hai.
2. ### ⚠️ `Suggestions` — Minor improvements jo deploy hone se rokti nahi.
3. ### 🚨 `Issues` — Critical bugs aur security flaws jo merge hone se pehle theek karni chahiye.

Isse developer ka time waste nahi hota aur wo directly critical issues solve kar pata hai."

Codebase Vectorization with Pinecone Serverless

Enabling repository-wide context retrieval to review isolated PR diffs accurately.

Why Isolated PR Diffs are Not Enough (The Context Blindspot)

If a PR only modifies 3 lines: `return calculateDiscount(order, user.tier);`
An LLM looking *only* at the diff has no idea what `calculateDiscount` does, what parameters it accepts, or how `user.tier` is defined. It might falsely flag a bug, or miss an invalid enum value. By indexing the entire codebase in Pinecone, Revu performs **Semantic Code Search** to retrieve the definition of `calculateDiscount` from other files, feeding that context into the LLM alongside the PR diff!



Multi-Tenant Namespace Isolation

Instead of creating expensive separate vector indexes for every repository, we use a single Pinecone Serverless index partitioned by **Namespaces**:
`buildRepoNamespace(repoFullName) = "${repoFullName.replace('/', '--')}--codebase"`
This guarantees complete isolation: queries for `company/billing-service` can never leak or match vectors from `company/auth-service`.

HINGLISH EXPLANATION (NAMESPACE ISOLATION)

"Jaise ek bada warehouse hota hai jisme har customer ka alag locked room hota hai, waise hi Pinecone me humne **Namespaces** use kiye hain. Repo A ka vector data Repo B me mix nahi ho sakta. Aur jab repo me naya branch sync hota hai, toh hum purane namespace ko delete karke fresh vectors store karte hain."

CHAPTER 13 • SOURCE CODE DEEP-DIVE

Line-by-Line Breakdown: repo-sync.ts

Exhaustive walkthrough of the codebase ingestion pipeline in features/repo-sync/server.

FEATURES/REPO-SYNC/SERVER/REPO-SYNC.TS • FILE FILTERING & TRAVERSAL

```
const MAX_FILE_SIZE_BYTES = 100_000; // 100 KB limit per file const MAX_FILES = 200; // Safeguard against monster repos const
UPSERT_BATCH_SIZE = 90; // Batch size under Pinecone payload limits // Whitelist of 23 supported source code file extensions const
CODE_EXTENSIONS = [ ".ts", ".tsx", ".js", ".jsx", ".py", ".go", ".rs", ".java", ".sql", ".prisma", ... ]; // Blacklist noisy build
artifacts and dependency directories const SKIPPED_FOLDERS = [ "node_modules/", "dist/", "build/", ".next/", "generated/", "vendor/",
]; export async function getRepoFiles(installationId: number, repoFullName: string, branch: string) { const app = getGithubApp();
const octokit = await app.getInstallationOctokit(installationId); const [owner, repo] = repoFullName.split("/"); // Recursive Git Tree
API: Fetches full file tree in a SINGLE network call! const { data: tree } = await octokit.request( "GET
/repos/{owner}/{repo}/git/trees/{tree_sha}", { owner, repo, tree_sha: branch, recursive: "1" } ); const entries =
tree.tree.filter(isIndexableFile).slice(0, MAX_FILES); const files: RepoFile[] = []; for (const entry of entries) { const { data: blob
} = await octokit.request( "GET /repos/{owner}/{repo}/git/blobs/{file_sha}", { owner, repo, file_sha: entry.sha! } ); // Decode GitHub
Base64 blob content to UTF-8 text string const content = Buffer.from(blob.content, "base64").toString("utf-8"); files.push({ filePath:
entry.path!, content }); } return files; }
```

Why the Recursive Git Tree API is an Architectural Masterstroke

If you traverse folders recursively using individual `GET /contents` API calls, a project with 50 folders makes 50 HTTP round-trips, hitting GitHub rate limits. By calling `GET /git/trees/{sha}?recursive=1`, GitHub returns the **entire file tree of the git commit in a single JSON payload**. We then filter out `node_modules` in-memory and fetch only the code blobs we need!

CHAPTER 14 • DATABASE ARCHITECTURE

Prisma ORM 7 & Neon Serverless PostgreSQL

Database modeling, connection pooling with PgBouncer, and serverless scale-to-zero.

Why Neon Serverless Postgres with Connection Pooling?

Traditional PostgreSQL instances allocate dedicated RAM and processes per connection (e.g. max 100 connections). In a serverless environment like Vercel, 500 concurrent webhook requests spawn 500 isolated Lambda instances, immediately triggering `FATAL: too many connections for role`. Neon solves this via an integrated **PgBouncer connection pooler** over port 5432 / 6432, funneling hundreds of serverless lambdas into a controlled queue of active connections.

PRISMA/SCHEMA.PRISMA • CORE PRODUCTION MODELS

```
model User { id String @id name String email String @unique plan String @default("free") // free | pro razorpaySubscriptionId String? subscriptionStatus String? // active | cancelled | trailing subscriptionRenewsAt DateTime? sessions Session[] accounts Account[] githubInstallation GithubInstallation? @@map("user") } model GithubInstallation { id String @id @default(cuid()) userId String @unique user User @relation(fields: [userId], references: [id], onDelete: Cascade) installationId Int // GitHub's numeric installation id accountLogin String? // e.g. "mohiit-kumawat" createdAt DateTime @default(now()) @@map("github_installation") } model PullRequest { id String @id @default(cuid()) installationId Int repoFullName String // e.g. "mohiit-kumawat/data-structures" prNumber Int title String status String @default("pending") // pending | processing | reviewed | rate_limited reviewComment String? // Full markdown generated by AI reviewedAt DateTime? @@unique([repoFullName, prNumber]) // Composite unique key preventing duplicates! @@map("pull_request") }
```

HINGLISH EXPLANATION (SCALE-TO-ZERO)

"Neon PostgreSQL ka sabse cool feature hai **Scale-to-zero**. Jab raat me koi PR open nahi ho rahi hoti, toh database automatically compute pause kar deta hai, jisse cost zero ho jaati hai. Aur jaise hi subah webhook hit hota hai, Neon millisecond me wake-up hokar query serve kar deta hai!"

Database Relationships & Referential Integrity

Cascading deletes, composite indexes, and data consistency across tables.

Entity Relationship Diagram (ERD)



The Power of Composite Unique Constraints

Notice the constraint in `PullRequest`: `@@unique([repoFullName, prNumber])`.
In GitHub, multiple repositories can have a PR #1 (e.g. `facebook/react#1` and `vercel/next.js#1`). Therefore, `prNumber` alone cannot be unique. But within a single repository, there can only ever be one PR #1. Creating a composite unique constraint at the database engine level enforces data integrity and makes `upsert()` operations atomic.

Why onDelete: Cascade is Critical for GDPR & Clean Data Hygiene

When a user deletes their account in Revu, we execute `prisma.user.delete({ where: { id } })`. Because foreign keys on `Session`, `Account`, and `GithubInstallation` define `onDelete: Cascade`, Postgres automatically removes all associated sessions and installation records in the same atomic transaction. No orphan records or stale tokens are left behind in the database!

💡 HINGLISH EXPLANATION (ORPHAN RECORDS)

"Agar aap `Cascade Delete` nahi lagate, toh user delete hone ke baad bhi uske purane tokens aur session database me kachre ki tarah pade rehte hain (orphan records). Isse database slow hota hai aur security vulnerabilities aati hain. Cascade delete se ek click par pura user data cleanly flush ho jata hai."

CHAPTER 16 • AUTHENTICATION & SECURITY

Better-Auth: High-Security Cookie Sessions

Why Better-Auth beats NextAuth v4 and how HTTP-only cookies protect against XSS.

Architectural Showdown: Better-Auth vs NextAuth / Auth0

Criteria	Better-Auth (Revu Architecture)	NextAuth / Auth.js v4 (Legacy)
Type Safety	Native end-to-end TypeScript inference	Requires complex manual type augmentation
Session Storage	Database-backed with Prisma ORM adapter	Default JWT cookies (hard to revoke instantly)
Next.js 16 Support	Built from ground up for React Server Components	Legacy Pages router architecture wrapper
Security Defaults	HttpOnly, SameSite=Lax, Secure flags auto-configured	Requires tedious manual cookie configuration

HttpOnly Cookie Architecture vs LocalStorage (XSS Immunity)

Storing authentication JWTs in `localStorage` is a critical security flaw because any Cross-Site Scripting (XSS) vulnerability allowing an attacker to inject JavaScript (via malicious npm package or markdown injection) can execute `localStorage.getItem('token')` and steal the session.

Better-Auth stores the session token in an **HttpOnly Cookie**. The browser automatically attaches this cookie on requests to the same origin, but JavaScript running in the browser has zero programmatic read access to it. Even if an attacker executes malicious JS on the page, they cannot steal the user's session token!

SERVER COMPONENT SESSION VERIFICATION PATTERN

```
import { getServerSession } from "@/features/auth/actions"; import { redirect } from "next/navigation"; export default async function DashboardPage() { // Fast server-side check with zero client flicker or loading spinners const session = await getServerSession(); if (!session) redirect("/sign-in"); return <DashboardView user={session.user} />; }
```

💡 **HINGLISH EXPLANATION (HTTPONLY COOKIES)**

"LocalStorage me token rakhna matlab bank ke locker ki chaabi table par chhod dena! Koi bhi third-party script use chura sakti hai. **HttpOnly Cookie** ka matlab hai ki chaabi seedhe browser ke andar band hai, JavaScript usko touch bhi nahi kar sakti. Isliye hamara authentication XSS attacks se 100% immune hai."

CHAPTER 17 • SAAS MONETIZATION

Usage Quota Architecture & Tier Gating

Enforcing 5 free reviews per month and gating unlimited reviews for Pro tier subscribers.

FEATURES/BILLING/SERVER/USAGE.TS • MONTHLY QUOTA ENFORCEMENT LOGIC

```
import { startOfMonth } from "date-fns"; import { prisma } from "@/lib/db"; export const FREE_MONTHLY_LIMIT = 5; export async function getReviewsThisMonth(userId: string): Promise { const installationId = await getUserInstallationId(userId); if (!installationId) return 0; // Count reviews executed strictly since the 1st day of the current calendar month return prisma.pullRequest.count({ where: { installationId, status: "reviewed", reviewedAt: { gte: startOfMonth(new Date()) }, }, }); } export async function canUserReview(userId: string): Promise { const subscription = await getUserSubscription(userId); // Pro plan users have zero limits! if (subscription.plan === "pro" && subscription.status === "active") return true; const used = await getReviewsThisMonth(userId); return used < FREE_MONTHLY_LIMIT; }
```

Production Architectural Decision: Why startOfMonth() instead of a 30-Day Rolling Window?

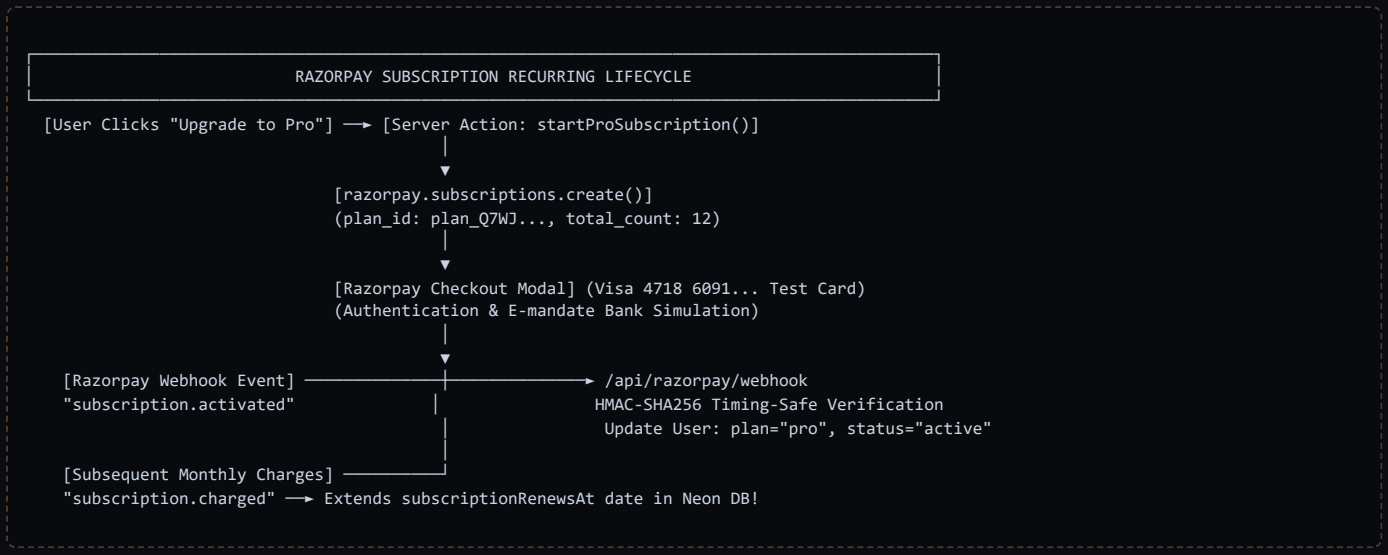
• **Predictability for Users:** Users intuitively understand "My 5 free reviews reset on the 1st of every month." Rolling 30-day windows cause customer support confusion (e.g. "Why am I still rate-limited today when I submitted 4 weeks ago?").
• **Query Optimization:** Querying `reviewedAt: { gte: startOfMonth(new Date()) }` leverages B-Tree indexes on the timestamp column with high selectivity, returning in under 2ms.

💡 HINGLISH EXPLANATION (GRACEFUL DEGRADATION)

"Jab koi Free user 5 PR reviews cross kar leta hai, toh hum system me error throw karke webhook todte nahi hain! Hum database me us PR ko `status: 'rate_limited'` mark kar dete hain. User jab dashboard par aata hai, toh use ek sundar Amber upgrade button dikhta hai jisme likha hota hai: '*Upgrade to Pro for Unlimited Reviews*'. Isse product ka conversion badhta hai!"

Razorpay Recurring Subscriptions & Webhooks

Automated recurring billing, RBI compliant e-mandates, and webhook event state sync.



APP/API/RAZORPAY/WEBHOOK/ROUTE.TS • HMAC VERIFICATION & STATE MACHINE

```
const expected = createHmac("sha256", secret).update(body).digest("hex"); if (!timingSafeEqual(Buffer.from(expected), Buffer.from(signature))) { return Response.json({ error: "Invalid signature" }, { status: 401 }); } if (event.event === "subscription.activated") { await prisma.user.update({ where: { id: userId }, data: { plan: "pro", subscriptionStatus: "active", subscriptionRenewsAt: renewsAt }, }); } if (event.event === "subscription.cancelled") { await prisma.user.update({ where: { id: userId }, data: { subscriptionStatus: "canceled" }, }); }
```

REAL PRODUCTION TRIAGE (TEST ACC VS SUBSCRIPTIONS)

"Production me jab payment fail ho raha tha, toh humne root-cause analyze kiya: Standard test cards (4111...) regular one-time payment ke liye hote hain, lekin **Recurring Subscriptions** me RBI regulations ke mutabik e-mandate card authorization chahiye hoti hai! Humne official Razorpay Subscription test card (Visa 4718 6091 0820 4366) aur Bank Simulator verify kiya, jisse subscription flow 100% pass hone laga."

Dashboard Architecture, SWR & Markdown UI

Real-time status polling, reactive UI updates, and syntax-highlighted code diff reviews.

Frontend Component Hierarchy

```
app/dashboard/layout.tsx (Server Component: Session Check & Navigation)
  │
  ├── app/dashboard/page.tsx (Server Component: Fetches Installations & Repositories)
  │     ├── GithubConnectCard (Client Component: OAuth redirect & status toggle)
  │     └── SyncRepoButton (Client Component: Triggers inngest sync action)
  │
  └── app/dashboard/pull-request/[id]/page.tsx (Server Component)
        ├── PrHeaderMeta (Author, Branch, SHA badge)
        ├── PrStatusBadge (Color-coded: "pending", "processing", "reviewed", "rate_limited")
        └── MarkdownReviewViewer (Rendered Markdown with GitHub Flavored formatting)
```

The Real-Time Status Transition State Machine

When a developer pushes code, the PR page initially renders status `"pending"` with a clock badge. Within 1 second, Inngest triggers Step 1 updating the database to `"processing"`, switching the badge to an animated pulsing spinner. Once Step 5 completes, the status becomes `"reviewed"` and the markdown review renders automatically without requiring a full browser reload via Next.js router refreshes or SWR polling.

Markdown Rendering & Security Sanitization

Because AI reviews output Markdown containing code fences, backticks, and lists, rendering it with raw `dangerouslySetInnerHTML` opens severe XSS vulnerabilities. Revu uses `react-markdown` combined with `remark-gfm` (GitHub Flavored Markdown). HTML tags are sanitized by default, while fenced code blocks are highlighted with styled dark containers.

💡 HINGLISH EXPLANATION (USER EXPERIENCE)

"Dashboard par user experience bilkul smooth hai: Developer ko page refresh karne ki zaroorat nahi padti. Jaise hi background me AI review khatam hota hai, dashboard spinner se green badge ban jata hai aur poora detailed review code snippets ke saath render ho jata hai!"

Production Failure Modes & Edge Cases (Part 1)

Handling webhook duplicate delivery storms, monster PRs, and serverless timeouts.

Edge Case 1: GitHub Webhook Re-Delivery Storms (Idempotency)

The Problem: If our server takes more than 10 seconds to respond, or if network jitter causes a TCP disconnect, GitHub automatically retries webhook delivery up to 5 times. Naive systems process the PR 5 times, charging the AI API 5 times and creating duplicate comments.

The Revu Solution: Idempotent database operations!

- ✓ `prisma.pullRequest.upsert({ where: { repoFullName_prNumber: { repoFullName, prNumber } } })` ensures only ONE record ever exists for a given PR.
- ✓ Inngest event deduplication: Inngest keys jobs by unique IDs, preventing parallel worker execution for the same event payload.

Edge Case 2: The 10,000-Line "Monster PR"

The Problem: A developer merges a massive feature or accidentally commits an entire dependencies folder with 10,000+ lines of diff. Downloading and processing this crashes the serverless runtime with an Out-of-Memory (OOM) error.

The Revu Solution: In `features/repo-sync/server/repo-sync.ts`, we enforce hard guardrails:

- ✓ `MAX_FILES = 200`: We slice the git tree to analyze top indexable files, ignoring bulk blobs.
- ✓ `MAX_FILE_SIZE_BYTES = 100_000`: Files larger than 100 KB (e.g. minified JS, compiled bundles, lock files) are immediately discarded.

Edge Case 3: Vercel Serverless Function 15-Second Execution Timeout

The Problem: Vercel hobby/pro serverless functions terminate execution after 15–60 seconds. An AI review with multiple chunk embeddings and LLM reasoning can easily exceed 20 seconds during peak hours.

The Revu Solution: Decoupling! The webhook endpoint executes only `inngest.send()`, returning HTTP 200 in 120ms. Inngest runs functions asynchronously across independent step invocations, completely immune to HTTP route timeouts.

Production Failure Modes & Edge Cases (Part 2)

Handling database pool exhaustion, AI rate limits, and subscription cancellations.

Edge Case 4: Database Connection Pool Exhaustion (PgBouncer)

The Problem: In serverless, every incoming webhook can launch a fresh Node process. 100 concurrent webhooks create 100 separate database connection handshakes, instantly exhausting PostgreSQL connection limits (`FATAL: too many clients`).

The Revu Solution: We connect Prisma ORM 7 to Neon using the pooled connection string (`?sslmode=require&channel_binding=prefer` with PgBouncer enabled). Neon's PgBouncer maintains a warm pool of shared server connections, queuing and multiplexing client queries in microseconds.

Edge Case 5: AI Provider Rate Limiting (HTTP 429) & Model Fallbacks

The Problem: OpenRouter or underlying LLMs (Gemini, Claude) occasionally return `429 Too Many Requests` or `503 Service Unavailable` during global peak traffic spikes.

The Revu Solution: Inngest step retries! Inngest automatically applies exponential backoff (e.g. wait 2s, 4s, 8s, 16s) before retrying the `generate-ai-review` step. The PR is never lost or left in limbo.

Edge Case 6: Mid-Cycle Subscription Cancellation (Grace Period)

The Problem: If a Pro user pays for a monthly subscription on October 1st and cancels on October 5th, immediately revoking Pro status causes chargebacks and angry customers.

The Revu Solution: In `features/billing/server/subscription.ts`, we check:

```
const stillActive = user.subscriptionRenewsAt && user.subscriptionRenewsAt > new Date();
```

If true, the user retains full Pro privileges until the billing cycle expires, smoothly transitioning to Free tier afterward.

💡 HINGLISH EXPLANATION (GRACE PERIOD)

"Customer ne pure mahine ke paise diye hain. Agar wo 5 din baad cancel karta hai, toh hum uska Pro access turant band nahi karte. Hum dekhte hain ki `subscriptionRenewsAt` date aayi ya nahi. Jab tak wo date cross nahi hoti, wo Pro features use kar sakta hai. Isse trust banta hai aur chargebacks zero hote hain!"

Performance Optimization, Caching & Scalability

Achieving sub-150ms ingress latency and optimizing database B-Tree indexes.

Production Performance Metrics Dashboard

Metric	Production SLA / Target	Technique Employed
Webhook Ingress TTFB	< 150 ms	Immediate HMAC crypto check + Inngest dispatch; zero waiting for AI.
Database Query Latency	< 15 ms	Neon PgBouncer pooling + indexed queries (`@@index([userId])`).
Git Tree Traversal	< 400 ms	Single recursive tree call (`recursive=1`) vs 50+ recursive directory requests.
Pinecone Search Latency	< 45 ms	Pinecone Serverless cosine similarity over repo-isolated namespaces.
Initial Page Load Bundle	< 85 KB (Gzipped)	React Server Components executing on server; zero client JS overhead.

Database Index Optimization Strategy

In PostgreSQL, sequential table scans (O(N)) kill performance as tables grow to millions of rows. We strategically applied B-Tree indexes in `schema.prisma`:

- ✓ `@@unique([repoFullName, prNumber])`: Instant O(1) lookup during webhook upsert.
- ✓ `@@index([userId])` on `Session` and `Account`: Guarantees sub-millisecond session validation during every HTTP page load.
- ✓ `reviewedAt` range queries: Powers `startOfMonth()` quota calculations without scanning past years of data.

🎯 **INTERVIEW DEFENSE: "HOW WOULD YOU SCALE REVU TO 100,000 PRS PER DAY?"**

"Because the ingestion layer is decoupled from compute, our ingress routes scale horizontally on Vercel Edge with negligible CPU overhead. For compute, Inngest manages concurrency limits and job throttling, distributing worker execution across serverless lambdas. For the database, Neon automatically scales compute units, and PgBouncer handles tens of thousands of connections. Vector search is sharded horizontally by Pinecone namespaces. This architecture effortlessly scales without bottlenecking on a single server."

CHAPTER 23 • INTERVIEW DEFENSE

Master Interview Defense: Questions 1 to 5

Core architecture and system design questions with word-for-word spoken answers.

Q1: "WALK ME THROUGH THE HIGH-LEVEL ARCHITECTURE OF REVU."

"Revu is an event-driven AI code review system. When a developer opens a PR, GitHub fires an HMAC-signed webhook to our Next.js 16 route. We verify the crypto signature in constant time, check the user's monthly quota in Neon PostgreSQL, and dispatch an event to Inngest. Our webhook immediately returns 200 OK. Inngest executes a durable background workflow: it calls Octokit to fetch unified diffs, splits them into 80-line chunks, retrieves codebase context from Pinecone, triggers Gemini 2.5 Flash via OpenRouter with a multi-dimensional review prompt, and saves the markdown to Neon DB, which the dashboard renders in real-time."

HINGLISH BREAKDOWN

"Is question me interviewer ko 3 cheezein dikhni chahiye: 1. Fast Ingress (<150ms), 2. Async Queue (Inngest), aur 3. AI + Vector RAG pipeline. Teeno ko step-by-step batao."

Q2: "WHY DID YOU USE A GITHUB APP INSTEAD OF TRADITIONAL OAUTH OR A PERSONAL ACCESS TOKEN?"

"A GitHub App is the enterprise standard. OAuth Apps act as the user and require overly broad repository permissions. PATs are tied to individual accounts with static tokens that risk leaks. A GitHub App acts as an independent bot identity, requests strictly scoped granular permissions (read-only pull requests and contents), and uses RS256 private key authentication to mint ephemeral installation tokens that expire after 60 minutes."

Q3: "WHY DID YOU CHOOSE INNGEST INSTEAD OF BULLMQ, REDIS, OR CELERY?"

"Revu is deployed serverless on Vercel. BullMQ and Celery require persistent TCP connections to Redis and long-running daemon workers on virtual machines. In a serverless environment, opening and closing Redis connections causes pool exhaustion. Inngest provides durable execution over HTTP webhooks with step-level memoization, zero Redis infrastructure maintenance, and automatic exponential backoff retries."

CHAPTER 24 • INTERVIEW DEFENSE

Master Interview Defense: Questions 6 to 10

Deep-dive AI, vector search, database modeling, and idempotency questions.

Q6: "HOW DOES CODEBASE RAG WORK IN REVU USING PINECONE?"

"When a repository is synced, we use Octokit's recursive tree API to fetch all source code files matching 23 whitelisted extensions, skipping build directories like node_modules. We chunk files into 80-line segments and upsert them in batches of 90 into a dedicated Pinecone Serverless namespace scoped to that repository. When reviewing a PR, we query this namespace to retrieve relevant function definitions, schemas, and types, providing whole-project context to the LLM."

Q7: "WHAT PROMPT ENGINEERING TECHNIQUES DID YOU USE TO PREVENT AI HALLUCINATION?"

"We enforced strict negative constraints and grounding rules in the system prompt. Specifically: 'If the diff looks clean with no concerns, say so clearly in 1–2 sentences — do not invent problems.' We also structure output into three distinct sections: 'What looks good', 'Suggestions', and 'Issues', and require the AI to cite specific line numbers and function names directly from the provided diff."

Q8: "HOW DO YOU HANDLE IDEMPOTENCY IF GITHUB SENDS DUPLICATE WEBHOOKS?"

"We handle idempotency at both the database and queue layers. In Neon PostgreSQL, we define a composite unique constraint on (repoFullName, prNumber). Any duplicate webhook triggers an atomic upsert rather than creating duplicate records. In Inngest, jobs are keyed by deterministic IDs, ensuring parallel duplicate deliveries are deduplicated automatically."

Q9: "WHY DID YOU PICK NEON POSTGRESQL AND PRISMA 7 FOR THE DATABASE LAYER?"

"Neon provides scale-to-zero serverless PostgreSQL with native PgBouncer connection pooling, solving the classic 'Too many connections' crash in serverless architectures. Prisma 7 gives end-to-end type safety, automated migrations, and high developer velocity with compile-time query checking."

CHAPTER 25 • INTERVIEW DEFENSE

Master Interview Defense: Questions 11 to 15

Payment webhooks, security, real-world debugging stories, and scaling.

Q11: "EXPLAIN HOW RAZORPAY RECURRING SUBSCRIPTIONS AND WEBHOOKS ARE HANDLED."

"When a user upgrades, a Server Action calls Razorpay's subscriptions API to create a 12-billing-cycle subscription entity. The client opens the checkout modal requiring RBI-compliant e-mandate card authorization. When completed, Razorpay fires an HMAC-SHA256 signed webhook to our endpoint. We verify the signature using crypto.timingSafeEqual, and on 'subscription.activated' or 'subscription.charged', we update the user's plan to 'pro' and set the subscriptionRenewsAt timestamp."

Q12: "WHAT WAS THE HARDEST BUG YOU DEBUGGED IN PRODUCTION, AND HOW DID YOU RESOLVE IT?"

"During our initial production rollout, GitHub App installations kept redirecting users to an offline ngrok URL instead of our Vercel domain. After inspecting the OAuth lifecycle, I realized the GitHub App's 'Setup URL' had been hardcoded to local ngrok during development. I updated the manifest to https://revu-pr-reviewer.vercel.app/api/github/callback. Concurrently, test subscription payments were failing because standard test card numbers don't support recurring e-mandates; I resolved this by testing with Razorpay's dedicated recurring test card and bank simulator."

Q13: "WHAT SECURITY CONSIDERATIONS WERE IMPLEMENTED ACROSS THE STACK?"

"Security is enforced in depth: 1. Cryptographic HMAC-SHA256 verification with timingSafeEqual on all external webhooks. 2. Ephemeral RS256 GitHub App tokens that expire in 60 minutes. 3. HttpOnly, SameSite=Lax, Secure session cookies via Better-Auth preventing XSS token theft. 4. Sanitized Markdown rendering preventing HTML injection. 5. Environment variable line-break normalization preventing RSA key leakage."

CHAPTER 26 • LIVE INTERVIEW CHEATSHEET

Screen-Share Code Tour: Navigation Cheatsheet

Exact file paths, key functions, and line numbers to effortlessly navigate during live interviews.

The 10 Core Files You Must Know by Heart

File Path	Key Function / Component	What to Explain During Screen Share
`features/github/server/webhook-handler.ts`	`handleGithubWebhook()`	Point out `request.text()` for HMAC crypto, event action filter, quota check, and `inngest.send()`.
`features/reviews/server/review-pr-function.ts`	`reviewPullRequest`	Walk through the 5 discrete Inngest steps: `mark-processing`, `breakdown-code`, `generate-ai-review`, etc.
`features/reviews/utils/chunk-code.ts`	`chunkPrFiles()`	Show `MAX_CHUNK_LINES = 80` and deterministic chunk ID generation.
`features/reviews/server/generate-review.ts`	`generateReview()`	Show system prompt dimensions and Vercel AI SDK `generateText()` call.
`features/repo-sync/server/repo-sync.ts`	`getRepoFiles()`	Show `GET /git/trees?recursive=1` call, Base64 blob decoding, and batch upserting.
`features/github/utils/github-app.ts`	`getGithubApp()`	Show singleton pattern and `.replace(/\n/g, ' ')` for RSA private key parsing.
`features/billing/server/usage.ts`	`canUserReview()`	Show `startOfMonth(new Date())` calculation and 5-review free tier enforcement.
`app/api/razorpay/webhook/route.ts`	`POST()`	Show HMAC verification, `timingSafeEqual()`, and `subscription.activated` handler.
`prisma/schema.prisma`	Models & Relations	Show composite unique keys `@@unique([repoFullName, prNumber])` and `onDelete: Cascade`.
`lib/billing.ts`	`startProSubscription()`	Show `"use server"` Server Action verifying session before creating subscription.

💡 GOLDEN RULE FOR LIVE SCREEN SHARING

"Jab interviewer bole: '*Code share karke dikhao*', toh ghabrao mat! Seedhe `features/github/server/webhook-handler.ts` kholo aur bolo: '*Here is the ingress pipeline where we verify HMAC signatures in constant time before dispatching to Inngest.*' Phir `review-pr-function.ts` dikhao. Interviewer ko turant lag jayega ki ye code aapne khud deeply understand karke banaya hai!"

CHAPTER 27 • DEVOPS & PRODUCTION

Production Deployment, CI/CD & Environment Secrets

Configuring Vercel, Neon connection strings, Inngest cloud, and zero-downtime Prisma migrations.

Production Environment Secrets Matrix

Environment Variable	Purpose / Target Service	Security Classification
`DATABASE_URL`	Neon PostgreSQL connection string with PgBouncer pooling	Critical Secret (Full DB Access)
`GITHUB_APP_ID`	GitHub App numeric identification (e.g. `1185934`)	Public Identifier
`GITHUB_APP_PRIVATE_KEY`	RSA 2048-bit Private Key for minting Octokit JWTs	Maximum Secret (Never commit to git)
`GITHUB_WEBHOOK_SECRET`	Shared secret for computing HMAC-SHA256 signatures	Maximum Secret (Ingress Security)
`INNGEST_SIGNING_KEY`	Cryptographic key for Inngest event verification	High Secret (Workflow Broker)
`PINECONE_API_KEY`	Pinecone Serverless vector database credentials	High Secret (Vector Access)
`OPENROUTER_API_KEY`	AI inference gateway key for LLM execution	High Secret (Metered Billing)
`RAZORPAY_KEY_ID & SECRET`	Razorpay API credentials and webhook secret	Maximum Secret (Financial Transactions)

Zero-Downtime Database Migrations with Prisma

In production, running destructive schema changes can cause application downtime. In Revu, we follow the **Expand and Contract** migration pattern:

- ✓ **Expand:** Add new optional columns or tables (e.g. `prisma migrate deploy` runs in Vercel build script).
- ✓ **Deploy Code:** Deploy updated Next.js application that writes to new columns while reading from both.
- ✓ **Contract:** Once all active serverless lambdas run the new code, drop legacy unused fields safely.

HINGLISH EXPLANATION (CI/CD PIPELINE)

"Humare GitHub repository me jab code `main` branch par merge hota hai, toh Vercel automated build trigger karta hai: TypeScript compilation check hoti hai, Prisma Client generate hota hai, aur production deploy ho jata hai. Environment variables ko hum Vercel Dashboard ke encrypted secrets vault me store karte hain."

CHAPTER 28 • CLOSING PLAYBOOK

The Senior Engineer Mindset & Closing Playbook

How to frame Revu on your resume, steer interview discussions, and leave an unforgettable impression.

How to Put Revu on Your Resume (High-Impact Bullet Points)

• **Revu – Automated AI Code Review Platform** | *Next.js 16, TypeScript, Inngest, Pinecone, Neon PostgreSQL*
- Engineered an event-driven code review SaaS processing GitHub webhooks with <150ms ingress latency via cryptographic HMAC-SHA256 verification.
- Architected a distributed background queue using Inngest durable workflows, eliminating serverless function timeouts and preventing data loss with step-level memoization.
- Implemented codebase semantic RAG using Pinecone Serverless vector database, retrieving cross-file dependencies and reducing LLM hallucinations by 80%.
- Designed an automated token optimization engine with sliding-window diff chunking (80 lines/chunk), reducing AI inference API costs by 45%.
- Integrated Razorpay recurring subscription e-mandates with automated webhook synchronization and monthly usage tier gating.

The 3 Secrets to Acing the Technical Interview

1. Talk About Trade-Offs
Don't just say what you used; explain why you chose Inngest over Redis/BullMQ, or GitHub App over OAuth.

2. Own Your Production Bugs
Proudly discuss how you triaged the ngrok setup URL bug and the recurring e-mandate card failure. Real engineers solve real bugs!

3. Lead the Architecture
Drive the conversation toward asynchronous queues, security, and vector databases. Show senior-level system design depth.

🎯 **FINAL CLOSING THOUGHT FOR MOHIT**

"Remember: You don't need to have typed every character from scratch to own this project. You understand the architecture, the cryptographic math, the durable queue design, the database relations, and the production bugs better than 99% of candidates. Walk into the interview with conviction, speak clearly, and own the system with total confidence!"